

รายงานทางเทคนิค — สถาปัตยกรรม Izara Anywhere

บทนำ

รายงานทางเทคนิคฉบับนี้จัดทำขึ้นเพื่ออธิบายสถาปัตยกรรม กระบวนการทำงาน

และการเชื่อมโยงของระบบ Izara Anywhere (อิสระ เทเลเมดิซีน)

ให้ผู้อ่านที่ไม่จำเป็นต้องเป็นนักพัฒนาสามารถติดตามได้เป็นฉบับเดียว โดยใช้ภาษาไทยเป็นหลัก

และอ้างอิงแผนภาพ

12 แท็บแรกจากไฟล์ docs/diagrams/diagrams.drawio เท่านั้น

หมายเหตุสำคัญ: ไฟล์ diagrams.drawio อาจมีมากกว่า 12 แท็บ (เช่น แท็บที่ 13 เกี่ยวกับการทดสอบ)

แต่รายงาน Word และ PowerPoint ฉบับนี้ตั้งใจไม่แก้ไขไฟล์แผนภาพ และไม่นำแท็บหลังจากลำดับที่

12 มาประกอบ

เพื่อให้เอกสารโฟกัสที่สถาปัตยกรรมและการใช้งานจริงของผู้ป่วย แพทย์ และผู้ดูแลระบบ

ลำดับการอ่าน: เริ่มจากภาพรวมพอร์ทัล → โมดูลทั้งแพลตฟอร์ม → ชั้นสถาปัตยกรรม →

การล็อกอิน → นวัตกรรม → ประชุมและ AI

→ การแจ้งเตือนแบบเรียลไทม์ → การ deploy → ข้อมูลสุขภาพ PHR/EMR → Living Will ตาม

PDPA → โครงสร้างฐานข้อมูล (ERD)

→ สรุปเส้นทาง End-to-End ทั้ง 12 ชั้น แต่ละข้อมีข้อความนำ คำอธิบายเชิงลึก ประเด็นสำคัญ

วิธีอ่านแผนภาพ และการเชื่อมต่อบทถัดไป

Izara Anywhere เป็นแพลตฟอร์มเทเลเมดิซีนสำหรับประเทศไทย ประกอบด้วย Patient Portal

(พอร์ทัลผู้ป่วย) Doctor Portal (พอร์ทัลแพทย์และผู้ดูแลระบบ)

และ Meeting Server (เซิร์ฟเวอร์ประชุมทางไกล) โดย deploy บน Google Cloud Run ภูมิภาค asia-southeast1

ฐานข้อมูลหลักคือ PostgreSQL 18 บน GCE VM การประชุมวิดีโอใช้ Jitsi (meet.jit.si)

การถอดความและสรุปทางคลินิกใช้ Gemini 3.1 Flash Lite

และระบบยึดหลัก Man-in-the-Loop (MITL) คือแพทย์ต้องตรวจสอบและลงนามก่อนที่ผลใด ๆ จะถึงมือผู้ป่วย

สรุปผู้บริหาร

สรุปผู้บริหาร: รายงานนี้อธิบาย 12 แผนภาพแรกของ Izara Anywhere เป็นภาษาไทยแบบต่อเนื่อง
ครอบคลุม (1) แผนที่พอร์ทัลและผู้ใช้

(2) โมดูลแพลตฟอร์มเต็มรูปแบบ (3) สถาปัตยกรรมชั้นงาน (4) การล็อกอินและสิทธิ์ (5)

กระบวนการนัดหมาย (6) pipeline ประชุมและ AI คลินิก

(7) เรียลไทม์และการแจ้งเตือน (8) การ deploy บน GCP (9) PHR/EMR และใบสั่งยา (10)

พินัยกรรมชีวิตตาม PDPA (11) ERD ฐานข้อมูล

(12) ขั้นตอน E2E ทั้งเส้นทาง

จุดเด่นทางสถาปัตยกรรม: ใช้ฐานข้อมูลกลางเดียว (single database) สำหรับทั้งสามพอร์ทัล วิดีโอ
WebRTC ไป Jitsi โดยตรงจากเบราว์เซอร์

การ sync คิวและแจ้งเตือนผ่าน PostgreSQL NOTIFY และ Socket.IO ผลจาก AI

อยู่ในสถานะร่างจนแพทย์ลงนาม

และข้อมูลอ่อนไหว (Living Will) มี versioning และ audit log ครบถ้วน

ผู้อ่านเป้าหมาย: ผู้บริหารโครงการ นักวิเคราะห์ระบบ ทีม QA/UAT แพทย์ที่ใช้งาน

และทีมพัฒนาที่ต้องการเอกสารอ้างอิงภาษาไทย

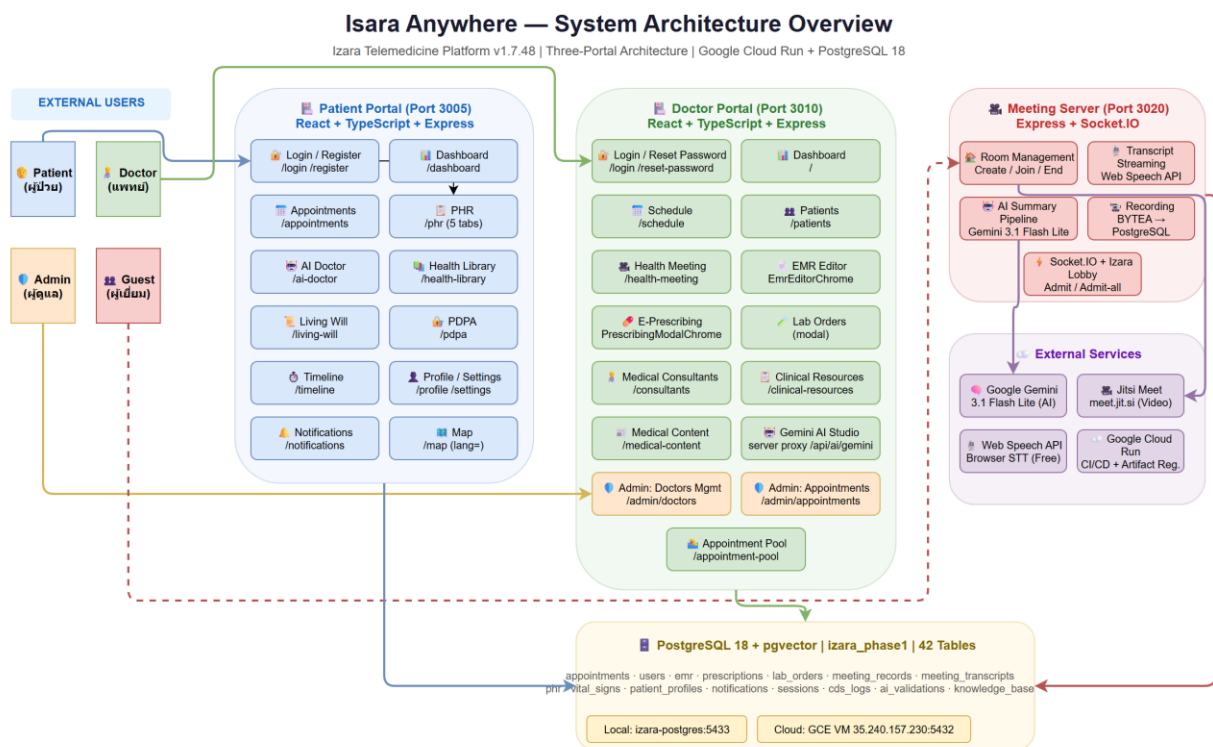
ที่สอดคล้องกับแผนภาพ โดยไม่ต้องเปิดไฟล์ draw.io ทุกครั้ง

บทที่ 1 — ภาพรวมระบบและแผนที่พอร์ทัล

บทเปิดของรายงานเริ่มจากมุมมองผู้ใช้งานจริง — ใครเข้าระบบจากช่องทางใด
บริการประชุมทางไกลอยู่ตรงไหน และข้อมูลสุขภาพรวมศูนย์ที่ได้ แผนภาพหน้านี้เป็น «แผนที่»
ก่อนลงรายละเอียดเทคนิคในบทถัดไป

วัตถุประสงค์ของบทนี้

อธิบายการจัดวางสามพอร์ทัลบนคลาวด์ ความสัมพันธ์ระหว่างผู้ป่วย แพทย์ ผู้ดูแลระบบ
และฐานข้อมูลกลาง เพื่อให้ทุกฝ่ายเห็นภาพเดียวกันก่อนอ่าน workflow ลึก
เหมาะสำหรับการอบรมผู้ใช้ใหม่และการนำเสนอต่อผู้บริหาร



ภาพที่ 1: 1. System Overview (Portal Map) (diagrams.drawio หน้า 1/12)

คำอธิบายและบริบท

Patient Portal (พอร์ทัลผู้ป่วย) เป็นจุดเข้าใช้งานหลักของผู้ป่วยและผู้ดูแล ให้บริการจองนัดหมาย
ดูประวัติสุขภาพส่วนบุคคล (PHR) ใ้หม้ไลน์เหตุการณ์สุขภาพ ปรีक्षा AI สุขภาพเบื้องต้น

ค้นหาแพทย์บนแผนที่ อ่านคลังเนื้อหาทางการแพทย์ จัดการเอกสาร PDPA และพินัยกรรมชีวิต (Living Will) รวมถึงดูการแจ้งเตือนเมื่อมีการมอบหมายนัดหรือผลการรักษา

Doctor Portal (พอร์ทัลแพทย์) ใช้โดยแพทย์และผู้ดูแลระบบ (Admin) แพทย์ใช้ดูคิวนัด บันทึก EMR/SOAP ออกใบสั่งยา สั่ง lab เข้าร่วมประชุมทางไกล ดูประวัติผู้ป่วยเมื่อมีสิทธิ์ และใช้ Gemini AI Studio สำหรับงานคลินิก ผู้ดูแลระบบใช้พอร์ทัลเดียวกันในบทบาท admin เพื่ออนุมัติบัญชีแพทย์ใหม่ จัดสรรนัดจาก Pool และดูภาพรวมระบบ

Meeting Server (เซิร์ฟเวอร์ประชุม) เป็นบริการกลางที่เชื่อม Jitsi จัดการ lobby (ห้องรอ) การ admit ผู้เข้าร่วม การบันทึกและถอดความ และ pipeline หลังจบประชุมเพื่อสรุป SOAP ร่างให้แพทย์ตรวจสอบ ผู้ป่วยและแพทย์ไม่ได้ «อยู่ใน» meeting server โดยตรง แต่เรียก API ของบริการนี้จากพอร์ทัลของตน

ทั้งสามพอร์ทัลอ่านและเขียนข้อมูลผ่าน PostgreSQL 18 บน schema izara_phase1 บน GCE VM ไม่แยกฐานข้อมูลต่อพอร์ทัล — ข้อดีคือข้อมูลนัดหมายและแจ้งเตือน sync ได้ทันที ข้อควรระวังคือทุกการเปลี่ยนแปลง schema ต้องพิจารณาผลกระทบทั้งสามฝ่าย

สื่อวิดีโอประชุมใช้ WebRTC จากเบราว์เซอร์ไปยัง Jitsi (meet.jit.si) โดยตรง Cloud Run ไม่ต้อง relay วิดีโอทั้งหมด จึงประหยัด bandwidth แต่ต้องตั้ง Permissions-Policy และ iframe ให้ camera/microphone ทำงานถูกต้อง มิฉะนั้นผู้ใช้จะเห็นหน้าจอดำหรือไม่มีเสียง

การ deploy บนคลาวด์ใช้ชื่อบริการ izara-patient-portal-dev-testing, izara-doctor-portal-dev-testing และ izara-meeting-server-dev-testing บน asia-southeast1 การพัฒนาในเครื่องใช้ docker-compose โดยพอร์ทัลหลักคือ 3005 (ผู้ป่วย) 3010 (แพทย์) และ 3020 (ประชุม)

หลัง deploy หรือแก้ไข configuration ควรรัน npm run cloud:smoke เพื่อตรวจ health ทั้งสามบริการ ก่อนให้ผู้ใช้ทดสอบจริง รายละเอียด URL และบัญชี demo อยู่ใน docs/markdown/operations/URLS_AND_DEFAULT_USERS.md

ประเด็นสำคัญ

- ผู้ป่วย: จองนัด → รอมอบหมาย/ยืนยัน → เข้าประชุม → ดูผลหลังแพทย์ลงนาม

- แพทย์: รับคิว → อ่าน PHR → ประชุม → แก่ SOAP → sign EMR → สั่งยา/lab
- แอดมิน: อนุมัติแพทย์ใหม่, มอบหมายจาก Pool, ดูภาพรวมนัด
- ฐานข้อมูลกลาง: PostgreSQL 18 บน GCE — ไม่แยก DB ต่อพอร์ทัล
- วิดีโอ: WebRTC → Jitsi โดยตรง ไม่ผ่าน Cloud Run เป็นหลัก
- ตรวจสอบสุขภาพบริการ: `npm run cloud:smoke`

พอร์ทัล	ผู้ใช้หลัก	หน้าที่หลัก
Patient Portal	ผู้ป่วย / ผู้ดูแล	จองนัด PHR แจ้งเตือน ประชุม PDPA
Doctor Portal	แพทย์ / แอดมิน	EMR คิว Pool อนุมัติบัญชี สั่งยา
Meeting Server	ระบบ (เรียกจากพอร์ทัล)	Jitsi lobby ถอดความ สรุป SOAP ร่าง

การอ่านแผนภาพ

ในแผนภาพ ให้สังเกตลูกศรจากผู้ใช้ (Patient / Doctor / Admin) เข้าสู่แต่ละพอร์ทัล และเส้นที่รวมลงกล่อง PostgreSQL ตรงกลาง — นี่คือหัวใจของสถาปัตยกรรม three-portal single database เส้นไป Jitsi มักแยกจากเส้น REST ไปพอร์ทัล แสดงว่าวิดีโอและข้อมูลธุรกิจเดินคนละทาง

เชื่อมต่อแบบทึบ

เมื่อทราบแล้วว่าใครใช้พอร์ทัลใดและข้อมูลรวมศูนย์ที่ไหน

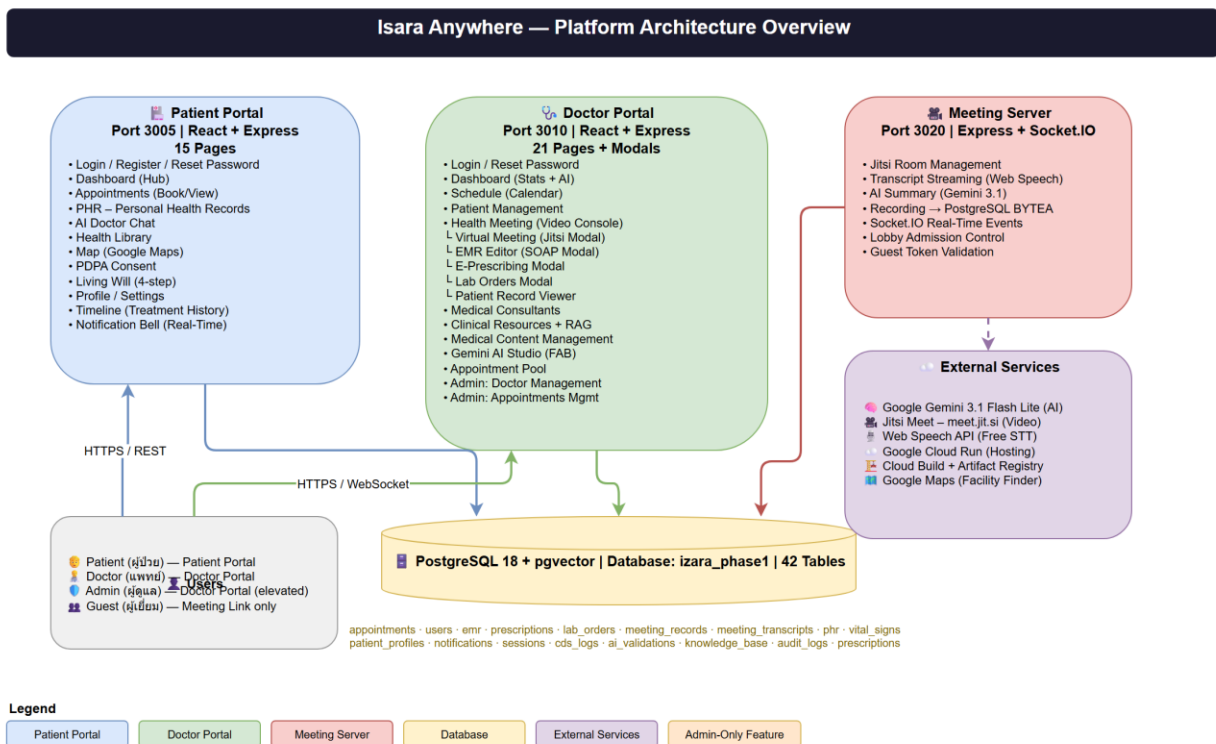
แบบทึบไปจะขยายรายละเอียดทุกโมดูลฟังก์ชันภายในแต่ละพอร์ทัลและบริการภายนอกที่เชื่อมต่อ

บทที่ 2 — แผนภาพแพลตฟอร์มเต็มรูป

ต่อจากแผนที่พอร์ทล บทนี้แสดงโมดูลย่อยทั้งหมดและเส้นทางข้อมูลระหว่าง Patient Portal, Doctor Portal, Meeting Server รวมถึงบริการภายนอก เช่น Gemini, Google Maps และ OAuth ช่วยให้เห็นว่าฟิเจอร์ Phase 1 อยู่ที่ไหนในโค้ดและ API ไດเชื่อมข้ามพอร์ทล

วัตถุประสงค์ของบทนี้

เป็นทะเบียนโมดูลสำหรับทีมพัฒนา ผู้ตรวจสอบ และทีม UAT เมื่อต้องทดสอบหรือแก้ defect จะรู้ได้ทันทีว่าหน้าจอใดอยู่พอร์ทลไหน และต้องตั้งค่า environment ไດให้ตรงกัน



ภาพที่ 2: 2. Full Platform Diagram (diagrams.drawio หน้า 2/12)

คำอธิบายและบริบท

ฝั่งผู้ป่วย (Patient Portal) ประกอบด้วย Appointments (จองและดูนัด) PHR (ประวัติสุขภาพ) Timeline (ไทม์ไลน์เหตุการณ์) AI Doctor (ปรึกษา AI เบื้องต้น) Map (ค้นหาแพทย์) Medical Content Library (คลังเนื้อหา) Living Will (พินัยกรรมชีวิต) และ Notifications (การแจ้งเตือน) ทุกหน้ามี route แยกใน React และเรียก REST API ภายใต้ /api/ ของ Patient Portal

ฝั่งแพทย์ (Doctor Portal) มี Dashboard สรุปงาน Queue คิวนัด Appointment Pool EMR Editor สำหรับ SOAP Prescribing ออกใบสั่งยา Lab Orders สั่งตรวจ Health Meeting เข้าประชุม Patient Record ดูข้อมูลผู้ป่วย Clinical Resources เอกสารอ้างอิง และส่วน Admin

สำหรับอนุมัติแพทย์และจัดสรรนัด แอดมินใช้ Doctor Portal ในบทบาท admin ไม่มีพอร์ทัลแยก

Meeting Server เปิด REST API สำหรับสร้างห้องประชุม จัดการ lobby ออก guest token และ postMeetingPipeline หลังจบประชุม Patient Portal และ Doctor Portal ต้องเรียก MEETING_SERVER_URL ที่ตั้งค่าเหมือนกันทุก environment หาก URL ไม่ตรงกัน ผู้ป่วยและแพทย์จะเข้าห้องคนละห้องหรือ lobby ค้าง

Doctor Portal บน production ใช้ unified image: nginx ฟังพอร์ต 8080 เซิร์ฟ static React และ proxy ไป API ภายใน (3011) พร้อม WebSocket Patient Portal build ด้วย Vite และรันใน container บน Cloud Run Meeting Server ใช้ Express + Socket.IO บนพอร์ต 3020 (local)

บริการภายนอก: Gemini 3.1 Flash Lite ใช้ผ่าน server-side proxy สำหรับ triage สรุป SOAP และ CDS Google Maps สำหรับค้นหาแพทย์ Google OAuth เมื่อเปิดใช้ SSO และช่องทางอีเมลแจ้งเตือนตามการตั้งค่าใน environment

แผนภาพนี้สอดคล้องกับโครงสร้างโฟลเดอร์ Isara-patient-portal, Isara-doctor-portal และ Isara-jitsi-server เมื่อเพิ่มฟีเจอร์ใหม่ควรอัปเดตแผนภาพและรายงานฉบับนี้พร้อมกัน

ประเด็นสำคัญ

- Patient: route แยกต่อโมดูล → REST /api/*
- Doctor: nginx :8080 → static + proxy API + WebSocket
- Meeting: REST + Socket.IO — URL ต้องตรงทุกพอร์ทัล
- Gemini: เรียกจาก server เท่านั้น ไม่ expose API key ฝั่ง client
- Admin ใช้ Doctor Portal บทบาท admin
- อ้างอิง Processes/Pages สำหรับขั้นตอนผู้ใช้แต่ละหน้า

การอ่านแผนภาพ

ติดตามกล่องสี่แต่ละพอร์ทและเส้น REST/WebSocket ข้ามกล่อง จุดที่มักผิดพลาดคือ Meeting URL ไม่ตรงกันระหว่าง patient และ doctor env และการลืม proxy WebSocket ใน nginx ทำให้ realtime ไม่ทำงาน

เชื่อมต่อบถัดไป

เมื่อเห็นโมดูลครบแล้ว บถัดไปจัดระเบียบเป็นชั้นสถาปัตยกรรม (layers)

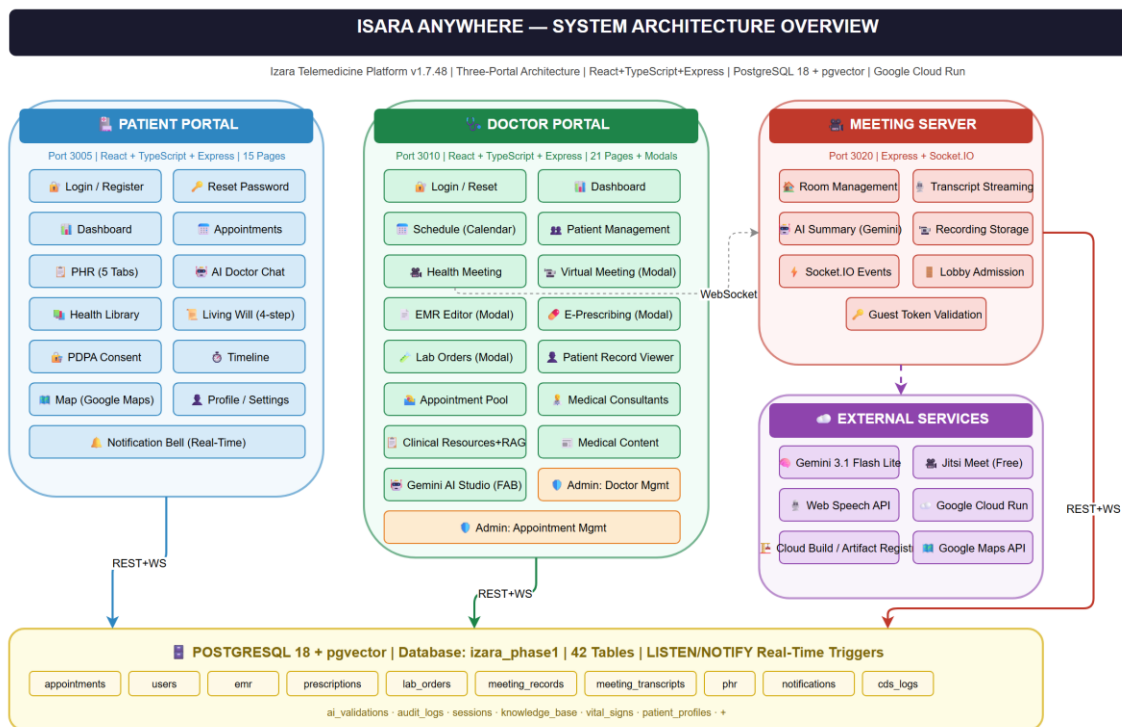
เพื่ออธิบายเทคโนโลยีที่ใช้ซ้ำในทุกพอร์ทและหลักความปลอดภัย

บทที่ 3 — สถาปัตยกรรมระบบ (ชั้นงาน)

จากแผนภาพแพลตฟอร์มเต็ม บทนี้จัดกลุ่มเป็นชั้นงานมาตรฐาน ทุก request และ event ในระบบต้องผ่านชั้นเหล่านี้ ช่วยให้ review ความปลอดภัย การ scale และการแก้ incident ทำได้เป็นระบบ

วัตถุประสงค์ของบทนี้

เป็นพื้นฐานอ้างอิงเมื่อออกแบบฟีเจอร์ใหม่ ตรวจสอบ PDPA หรืออธิบายให้หน่วยงานกำกับดูแล ทิมสามารถถามว่า «ฟีเจอร์นี้อยู่ชั้นใด ใช้ข้อมูลอะไร ส่งออกไปที่ไหน» ได้ชัดเจน



ภาพที่ 3: 3. System Architecture (diagrams.drawio หน้า 3/12)

คำอธิบายและบริบท

ชั้น Client (ไคลเอนต์): React 18 + TypeScript + Vite แสดง UI ทั้งสองพอร์ทัล ฝัง Jitsi ผ่าน iframe ใช้ Web Speech API สำหรับถอดความฟังเบราร์เซอร์เมื่อเปิดใช้ ไม่มี native mobile app ใน Phase 1 — ออกแบบ responsive สำหรับเบราร์เซอร์

ชั้น Portal (แอปพลิเคชัน): Node.js + Express รับ REST และ WebSocket Patient ใช้ session/cookie หรือ token แพทย์ใช้ JWT middleware ตรวจสอบ role และ rate limit ทุก route ที่เข้าถึงข้อมูลคลินิกต้องผ่าน RBAC

ชั้น Real-time (เรียลไทม์): เมื่อมีการ UPDATE ตารางสำคัญ (เช่น appointments) PostgreSQL TRIGGER ส่ง NOTIFY ไป listener ใน server listener แปลงเป็น Socket.IO event ไปยัง client ทำให้คิวและแจ้งเตือนอัปเดตโดยไม่ refresh

ชั้น Data (ข้อมูล): PostgreSQL 18 + pgvector สำหรับ knowledge_base.embedding Phase 1 ไม่เก็บไฟล์คลินิกหลักบน GCS — เน้นข้อมูล relational และ JSONB Schema หลักอยู่ใน scripts/database/izara-database.sql

ชั้น External (ภายนอก): Gemini 3.1 Flash Lite เรียกผ่าน proxy ฝั่ง server Google Maps API Google OAuth และบริการอีเมลตาม config API key เก็บใน Secret Manager หรือ .env ที่ไม่ commit

หลัก MITL (Man-in-the-Loop): ทุกผล AI ทางคลินิกอยู่ในสถานะ draft จนแพทย์ sign หรือ validate ผู้ป่วยเห็นเฉพาะขั้นที่ API filter แล้ว ไม่แสดง differential diagnosis ดิบหรือบันทึกร่างที่ยังไม่ลงนาม

ความปลอดภัย: Helmet, CORS จำกัดบน Cloud Run, bcrypt สำหรับรหัสผ่าน audit_logs บันทึกการเข้าถึง PHR และ Living Will consent ตาม PDPA

ประเด็นสำคัญ

- Client: React + Jitsi iframe + Web Speech (ถ้าเปิด)
- Portal: Express + JWT/session + RBAC
- Real-time: NOTIFY → Socket.IO (ไม่พึ่ง polling อย่างเดียว)
- Data: PostgreSQL 18 + pgvector บน GCE
- External: Gemini 3.1, Maps, OAuth — key ฝั่ง server
- MITL: draft จนแพทย์ sign — ผู้ป่วยเห็นเฉพาะที่ filter

ชั้น	เทคโนโลยี	หมายเหตุ
Client	React 18, Vite, Jitsi	เบราว์เซอร์เท่านั้น Phase 1
Portal	Express, nginx	Doctor Cloud ฟัง :8080
Real-time	NOTIFY + Socket.IO	sync คิวและแจ้งเตือน
Data	PostgreSQL 18	schema izara_phase1
External	Gemini 3.1, Maps	Secret Manager / env

การอ่านแผนภาพ

อ่านจากบนลงล่าง: Client → Portal → (Real-time + Data) → External

ลูกศรลงแสดงการฟังพา ลูกศรขึ้นแสดง response หรือ event กลับ กล้อง MITL/PDPA มักอยู่ข้ามชั้น

Portal และ Data

เชื่อมต่อบทถัดไป

ก่อนเข้าสู่ workflow ธุรกิจ ผู้ใช้ทุกคนต้องผ่านการยืนยันตัวตน — บทถัดไปอธิบาย login, session

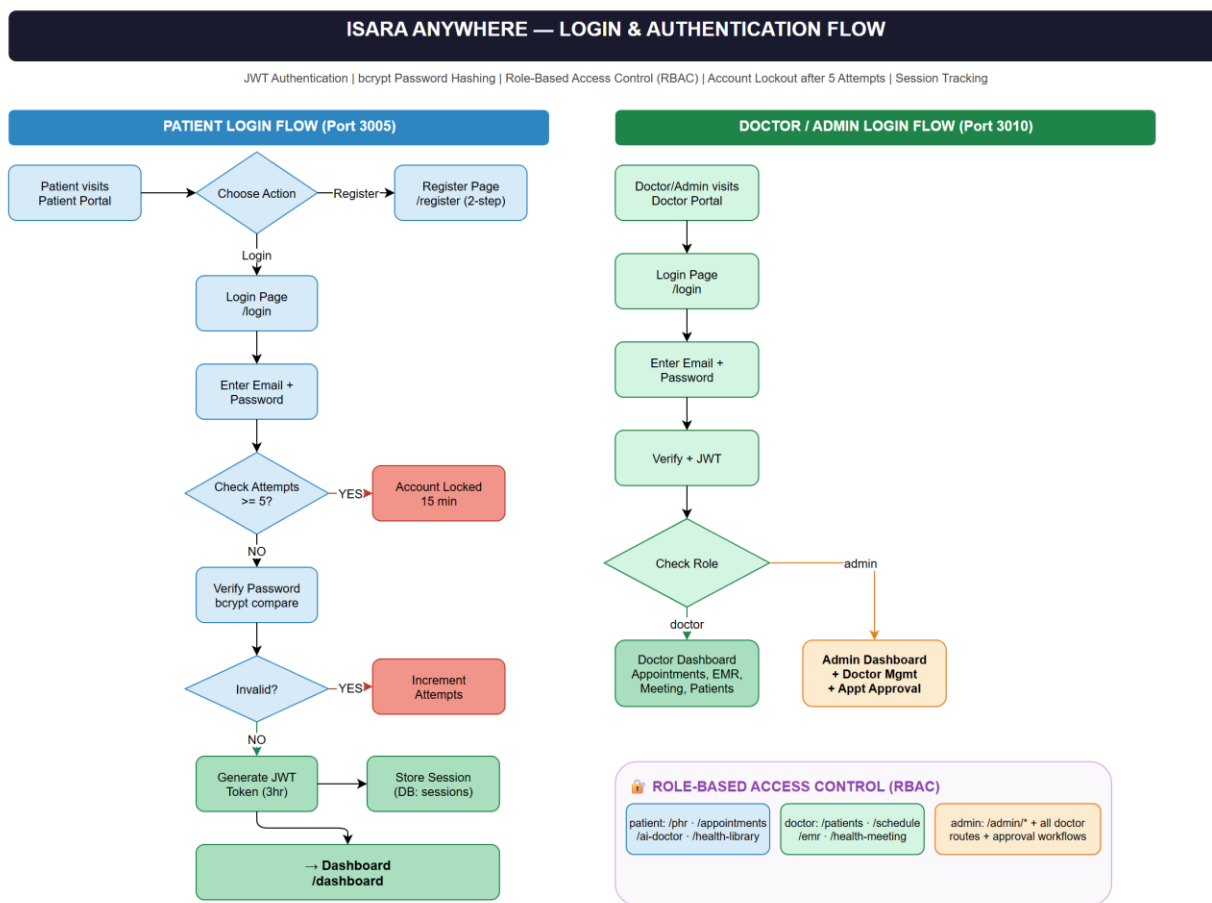
และการอนุมัติแพทย์ใหม่

บทที่ 4 — การล็อกอินและการพิสูจน์ตัวตน

สืบจากชั้น Portal การยืนยันตัวตนแยกบทบาท patient, doctor และ admin และกำหนดสิทธิ์ก่อนเข้า route ลึกที่เกี่ยวข้องข้อมูลคลินิก แผนภาพนี้ตอบคำถาม «ใครล็อกอินที่ไหน token อยู่ที่ได้ และแพทย์ใหม่ใช้งานได้เมื่อไหร่»

วัตถุประสงค์ของบทนี้

อธิบาย flow การเข้าระบบสำหรับทีม support ทีมความปลอดภัย และผู้ทดสอบ UAT ลดความสับสนระหว่างพอร์ทัลผู้ป่วยและแพทย์ และชี้จุดที่ต้องอนุมัติแอดมิน



ภาพที่ 4: 4. Login & Auth Flow (diagrams.drawio หน้า 4/12)

คำอธิบายและบริบท

ผู้ป่วยล็อกอินผ่าน Patient Portal: ส่ง POST login ด้วยอีเมล/รหัสผ่าน ระบบสร้าง session หรือ token เก็บใน cookie/header รองรับ Google SSO เมื่อตั้ง VITE_GOOGLE_CLIENT_ID — อีเมลจาก Google ต้องตรงกับ users.email ที่ลงทะเบียนแล้ว มิฉะนั้นจะ login ไม่สำเร็จ

แพทย์และแอดมินล็อกอินผ่าน Doctor Portal: ตรวจสอบรหัสผ่านด้วย bcrypt ออก JWT อายุตามการตั้งค่า ทุก /api ตรวจสอบ role (doctor หรือ admin) แพทย์ที่สมัครใหม่มี is_approved=false จนแอดมินกดอนุมัติที่ Manage Doctors จึงจะเห็นคิวและ EMR ได้ครับ

ตาราง users เก็บ role, login_attempts, locked_until, approval_status sessions เก็บ token และ expires_at สำหรับ revoke นโยบายรหัสผ่าน: อย่างน้อย 12 ตัวอักษร — อย่าใช้รหัส demo บน production

การเข้า PHR ของผู้ป่วยโดยแพทย์ต้องมี active appointment ที่เกี่ยวข้อง มิฉะนั้น RBAC บล็อกและบันทึก audit_logs Living Will แพทย์เห็นได้เฉพาะเมื่อผู้ป่วยเปิด is_shared_with_doctors

Demo users และ URL สำหรับทดสอบ:

docs/markdown/operations/URLS_AND_DEFAULT_USERS.md แผนภาพแสดง branch แยก

Patient login vs Doctor login และจุดรออนุมัติแพทย์

ประเด็นสำคัญ

- ผู้ป่วย: session/token + Google SSO (ถ้าเปิด)
- แพทย์: JWT + bcrypt + ตรวจสอบ role ทุก API
- แพทย์ใหม่: รอ admin อนุมัติที่ Manage Doctors
- PHR โดยแพทย์: ต้องมีนัด active + audit log
- รหัสผ่าน: ≥12 ตัวอักษร, ไม่ใช่ demo บน prod
- ล็อกบัญชี: login_attempts และ locked_until

การอ่านแผนภาพ

ติดตามเส้นทางจากหน้า Login แยก Patient vs Doctor ไปยัง Dashboard สังเกตจุด branch «รออนุมัติ» สำหรับแพทย์ใหม่ และเส้นไป sessions/users ในฐานะข้อมูล

เชื่อมต่อบทถัดไป

เมื่อผู้ใช้เข้าระบบและมีสิทธิ์ครบแล้ว กระบวนการหลักคือการจองและดำเนินการนัดหมาย —
บทถัดไปอธิบาย state machine ของ appointments

บทที่ 5 — กระบวนการนัดหมาย

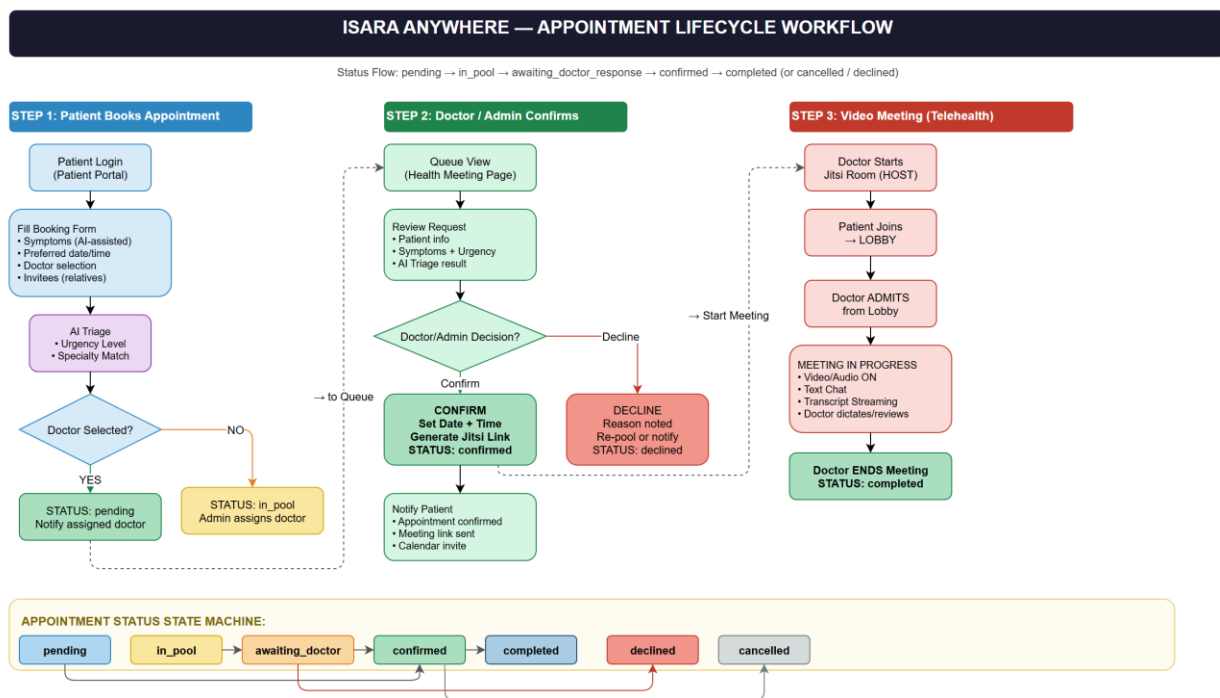
ต่อจาก auth การจองนัดเชื่อมผู้ป่วย แพทย์ แอดมิน และการ sync แบบ realtime

เป็นหัวใจของเทเลเมดิซิน — ตั้งแต่กรอกอาการจนถึงเข้าประชุมและปิดงาน

วัตถุประสงค์ของบทนี้

อธิบายสถานะนัดหมายทั้งหมด ใครเปลี่ยนสถานะเมื่อใด การแจ้งเตือนใดเกิดขึ้น และจุดที่มักเกิด defect

ในการ sync คิว — เหมาะสำหรับ UAT และ support



ภาพที่ 5: 5. Appointment Workflow (diagrams.drawio หน้า 5/12)

คำอธิบายและบริบท

ผู้ป่วยเริ่มจอง: กรอกอาการ (symptoms เก็บเป็น JSONB) เลือกประเภทและความเร่งด่วน ระบบ INSERT appointments สถานะเริ่ม pending หรือ in_pool ขึ้นกับการเลือกแพทย์โดยตรงหรือเข้า Pool ให้แอดมิน/แพทย์รับ

แอดมินหรือแพทย์รับจาก Pool: กำหนด doctor_id สถานะเปลี่ยนเป็น assigned ระบบ INSERT notifications แจ้งทั้งผู้ป่วยและแพทย์ แพทย์ยืนยัน → confirmed → เมื่อถึงเวลานัด in_meeting → หลังจบ completed

ทุก transition สำคัญ trigger PostgreSQL NOTIFY → Socket.IO Dashboard และ Queue ของแพทย์อัปเดตโดยไม่ต้อง refresh หน้า v1.7.48 เพิ่ม PUT read-all สำหรับ notifications แก้ปัญหา isRead ไม่ sync

meet_link และ jitsi_room_name สร้างเมื่อเข้าสู่ช่วงประชุม รายละเอียดขั้นตอนผู้ใช้: Processes/Appointment_Workflows.md และ screenshot กลุ่ม D ใน docs/screenshots/group-D/

สถานะที่ยกเลิก: cancelled — ต้องแจ้งเตือนทั้งสองฝ่ายและไม่ให้เข้าประชุม Admin Pool แสดงนัดที่รวมอบหมาย Doctor Queue แสดงนัดที่รับแล้ว

ประเด็นสำคัญ

- สถานะ: pending → in_pool → assigned → confirmed → in_meeting → completed
- Pool: แอดมิน/แพทย์มอบหมาย + แจ้งเตือน
- Realtime: NOTIFY → Socket.IO อัปเดตคิว
- read-all notifications: v1.7.48
- Jitsi room: สร้างเมื่อ in_meeting
- อ้างอิง screenshot D07-D16 สำหรับ UAT

ชั้น	ผู้ดำเนินการ	ผลบนระบบ
จอง	ผู้ป่วย	appointments + notification
รับจาก Pool	แอดมิน/แพทย์	doctor_id, assigned
ยืนยัน	แพทย์	confirmed
ประชุม	ทั้งสองฝ่าย	in_meeting, Jitsi
ปิดงาน	แพทย์/ระบบ	completed, EMR

การอ่านแผนภาพ

ตามลูกศรสถานะจากซ้ายไปขวา สังเกตจุดแยก Pool vs จองตรงแพทย์ และจุด NOTIFY ไปยัง Notification / Socket.IO

เชื่อมต่อบทถัดไป

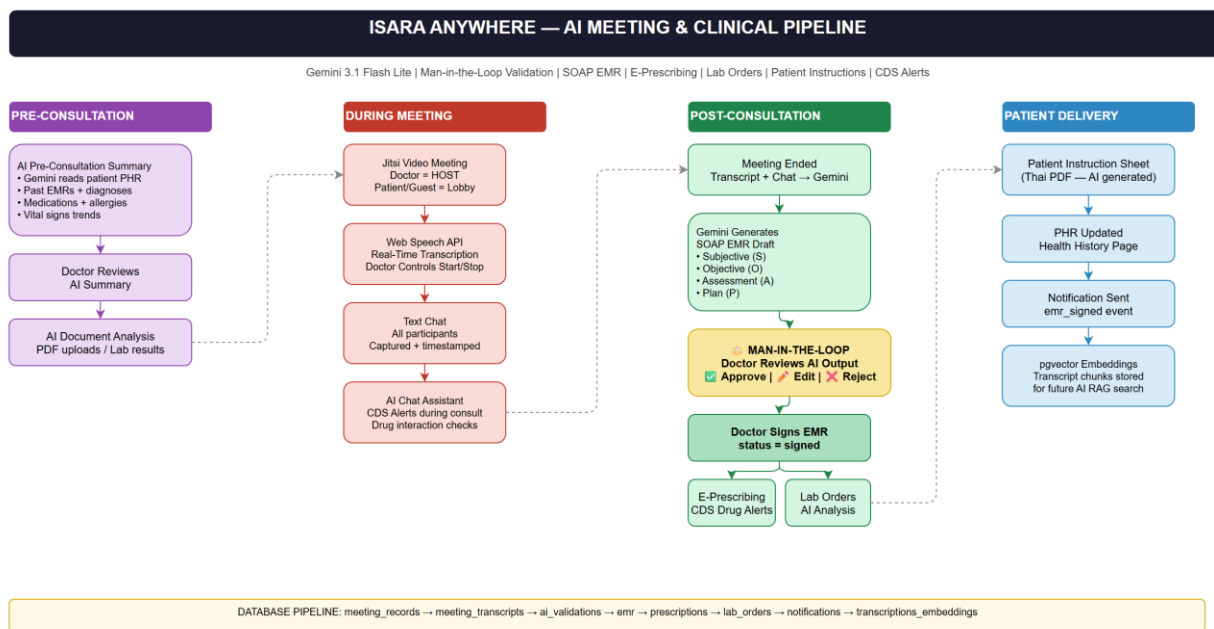
เมื่อนัดเป็น in_meeting ระบบประชุมและ AI เริ่มทำงาน — บทถัดไปคือ pipeline วิดีโอ ถอดความ และ SOAP

บทที่ 6 — Pipeline การประชุมและ AI คลินิก

จาก in_meeting ใน workflow นัดหมาย บทนี้อธิบายเส้นทาง Jitsi การบันทึก การถอดความ และการสรุป SOAP ร่างให้แพทย์ — เน้นหลัก MITL ว่าผู้ป่วยไม่ได้รับผล AI โดยตรง

วัตถุประสงค์ของบทนี้

ชี้แจงการทำงานร่วมกันระหว่าง Jitsi, Meeting Server และ Gemini สำหรับแพทย์ ทีม QA และผู้ตรวจสอบด้านคลินิก



ภาพที่ 6: 6. AI Meeting Pipeline (diagrams.drawio หน้า 6/12)

คำอธิบายและบริบท

แพทย์เป็น HOST ใน Jitsi — ผู้ป่วยและ guest รอที่ lobby จนได้รับ admit (Izara lobby M2) ป้องกันคนแปลกหน้าเข้าห้องโดยไม่ได้รับอนุญาต

ระหว่างประชุม: MediaRecorder บันทึกเสียง/วิดีโอ (เข้ารหัส AES-GCM ตามการตั้งค่า) Web Speech หรือ Google STT เติม meeting_transcripts แบบ real-time แพทย์เห็น LiveTranscriptionView บนหน้าประชุม

หลังประชุม: postMeetingPipeline ส่ง transcript ไป Gemini 3.1 ได้ ai_summary เก็บใน meeting_records (JSONB) — ยังเป็นร่าง

แพทย์เปิด EMR Editor (EmrEditorChrome) แก้ไข SOAP draft ตรวจสอบถูกต้อง แล้ว sign → status=signed เท่านั้นที่ถือเป็นบันทึกทางการ

จาก EMR signed: ออก prescriptions (มี CDS ตรวจยา) และ lab_orders แจ้งผู้ป่วยเมื่อพร้อม ผู้ป่วยดู timeline และ instruction sheet ภาษาไทยง่าย

Guest join ใช้ token + lobby API บน meeting server Recording แสดงบน dashboard หลังประชุม (ทดสอบกลุ่ม Q ใน screenshot)

ประเด็นสำคัญ

- HOST: แพทย์ — ผู้ป่วย/guest รอ lobby
- ถอดความ: Web Speech / STT → meeting_transcripts
- สรุป AI: Gemini 3.1 → ai_summary (ร่าง)
- MITL: แพทย์ sign EMR ก่อนส่งผู้ป่วย
- CDS: ตรวจยาตอน prescribing
- อ้างอิง Processes/VIDEO_MEETING_JITSI_GEMINI.md

การอ่านแผนภาพ

ติดตามลำดับ: Join Jitsi → Record/STT → AI Summary → Doctor Edit → Sign → Patient view เส้นที่ข้ามไป Patient ควรอยู่หลัง Sign เท่านั้น

เชื่อมต่อบทถัดไป

ขณะประชุมและเปลี่ยนสถานะนัด ระบบต้อง push event ทันที — บทถัดไปอธิบาย NOTIFY และ notifications

บทที่ 7 — Realtime และการแจ้งเตือน

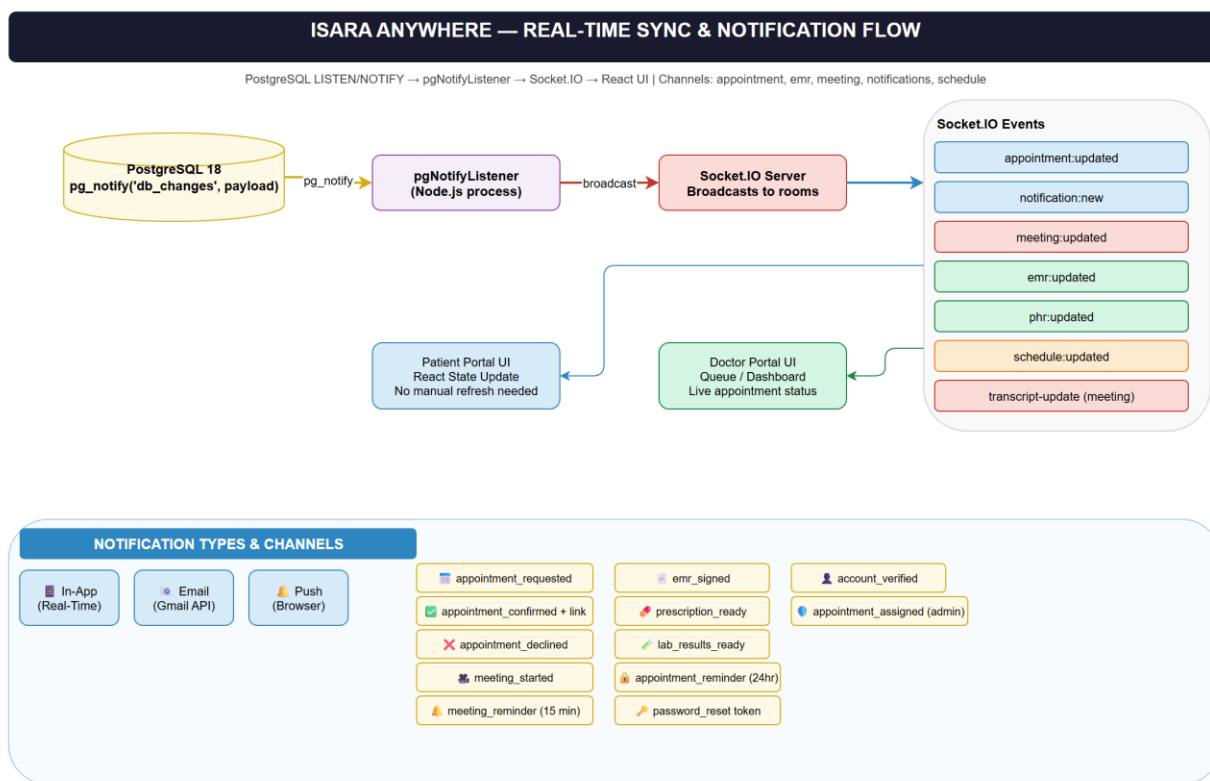
เชื่อม appointment และ meeting กับ UI ที่อัปเดตทันทีและระบบแจ้งเตือนสองพอร์ทัล

อธิบายว่าทำไมคิวและกระดิ่งแจ้งเตือนถึง sync หลัง assign โดยไม่ refresh

วัตถุประสงค์ของบทนี้

เป็นคู่มือสำหรับ debug ปัญหา «คิวไม่อัปเดต» หรือ «อ่านแล้วแต่ยังมี badge» และอธิบาย API

อ่าน/อ่านทั้งหมด



ภาพที่ 7: 7. Realtime & Notifications (diagrams.drawio หน้า 7/12)

คำอธิบายและบริบท

เมื่อ UPDATE appointments (เช่น doctor_id, status) TRIGGER ใน PostgreSQL ส่ง NOTIFY ไป channel ที่ pgNotifyListener ใน server ฟังอยู่

Listener แปลงเป็น Socket.IO event ไป client React อัปเดต state ใน Dashboard และ Queue

notifications INSERT เมื่อจอง มอบหมาย ยืนยัน หรือมีผลคลินิก ผู้ใช้เปิด NotificationsPage หรือ NotificationBell / DoctorNotificationBell

normalizeNotificationRow: isRead = Boolean(read_at) ทั้ง patient และ doctor data service ต้องใช้ logic เดียวกันเพื่อไม่ให้ badge ไม่ตรงกัน

PUT .../notifications/:userId/read-all อัปเดต read_at ทุกแถวของ user ครั้งเดียว ลด N+1 request และแก้ defect isRead v1.7.48

notificationRouting แยกประเภท appointment / system / clinical ตาม type และ data JSONB ไม่พึ่ง polling อย่างเดียว — มี Socket.IO เป็นหลักสำหรับ sync คิว

ประเด็นสำคัญ

- NOTIFY → pgNotifyListener → Socket.IO
- หน้า: /notifications (patient), DoctorNotificationBell
- isRead = มี read_at (normalize ทั้งสองพอร์ทัล)
- read-all API: v1.7.48
- ประเภท: appointment, system, clinical
- Defect DN*: notification routing และ read parity

การอ่านแผนภาพ

ดูเส้นจาก PostgreSQL NOTIFY ไป Socket.IO ไป UI และเส้น REST แยกสำหรับ GET notifications และ PUT read-all

เชื่อมต่อบทถัดไป

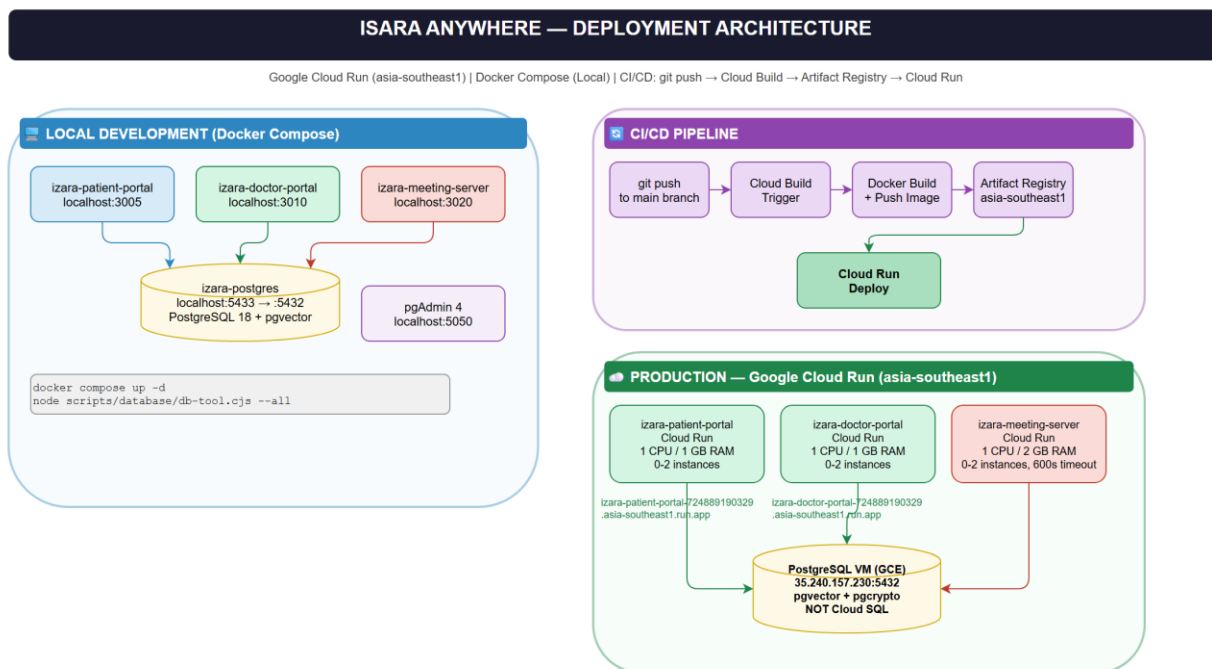
เมื่อเข้าใจ flow บนคลาวด์แล้ว บทถัดไปแสดงวิธี deploy และเปรียบเทียบ local vs production

บทที่ 8 — สถาปัตยกรรมการ deploy

ต่อจาก realtime บทนี้เปรียบเทียบกับ Docker Compose บนเครื่องพัฒนากับ Cloud Run production และเส้นทาง CI/CD — เป็น runbook สรุปสำหรับ release และ incident

วัตถุประสงค์ของบทนี้

ให้ DevOps และนักพัฒนาเห็นว่า service ใดอยู่ที่ไหน port ใด secret ใดสำคัญ และคำสั่งตรวจหลัง deploy



ภาพที่ 8: 8. Deployment Architecture (diagrams.drawio หน้า 8/12)

คำอธิบายและบริบท

Local: `docker compose up -d` — postgres :5433, patient :3005, doctor :3010, meeting :3020, pgAdmin :5050 seed ข้อมูล: `node scripts/database/db-tool.cjs --seed`

Production: โปรเจกต์ GCP izara-telemedicine region asia-southeast1 Cloud Run services ชื่อ *-dev-testing (หรือตาม tag release)

CI/CD: git push → Cloud Build (cloudbuild.yaml) → Artifact Registry → gcloud run deploy

Secrets: JWT, DATABASE_URL, GEMINI_API_KEY, GOOGLE_CLIENT_ID ผ่าน Secret Manager หรือ env MEETING_SERVER_URL ต้องตรงกันทุก portal .env — มักเป็นสาเหตุ lobby ค้าง

DB: GCE VM PostgreSQL ไม่ใช่ Cloud SQL — ต้องเปิด firewall เฉพาะ egress Cloud Run

Doctor image unified: nginx ฟัง 8080 บน Cloud Run

หลัง deploy: npm run cloud:smoke, verify:cloud-meeting-ai, TEST_ENV=cloud Playwright

ประเด็นสำคัญ

- Local ports: 3005 / 3010 / 3020 / 5433
- Cloud: *.run.app ทั้งสามบริการ
- DB: GCE VM — ไม่ใช่ Cloud SQL
- MEETING_SERVER_URL ต้องตรงทุกที่
- ตรวจสอบ: cloud:smoke, verify:cloud-meeting-ai
- อย่า commit .env หรือ service account key

สภาพแวดล้อม	Patient	Doctor	Meeting
Local (Docker)	:3005	:3010	:3020
Cloud Run	*.run.app	*.run.app	*.run.app

การอ่านแผนภาพ

ซ้ายมักเป็น Local Docker ขวาเป็น GCP — ติดตามลูกศร build/deploy และเส้น DB รวมศูนย์ที่ GCE

เชื่อมต่อจบถัดไป

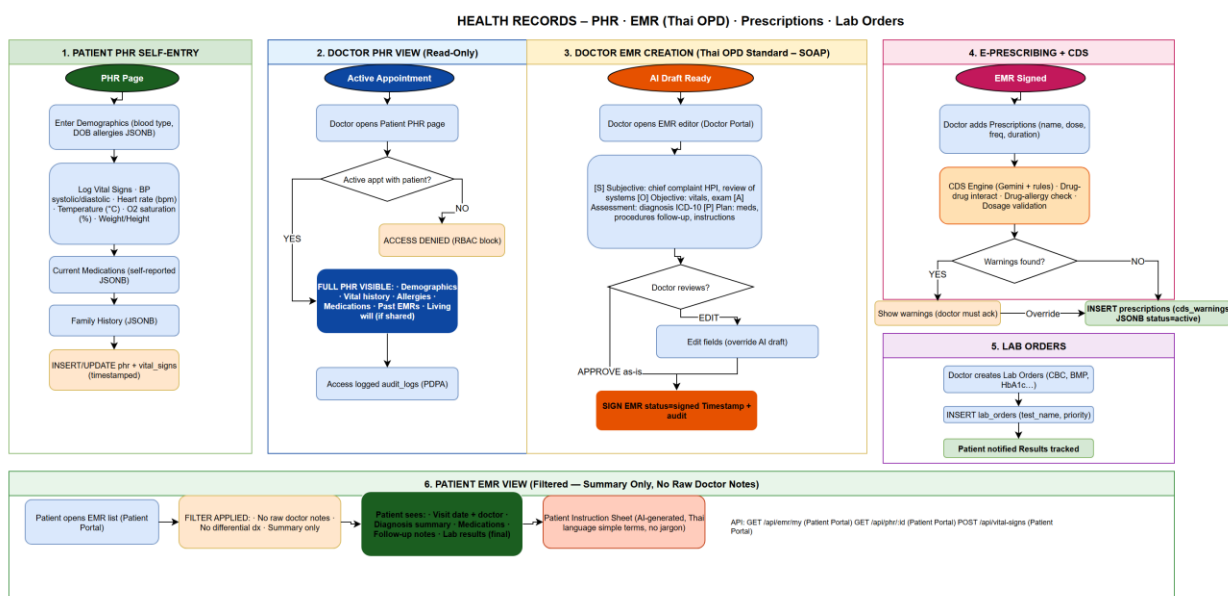
โครงสร้างที่ deploy แล้วรองรับข้อมูลสุขภาพ — ขนส่งไปลงลิ้งค์ PHR EMR ใบสั่งยา และ lab

บทที่ 9 — บันทึกสุขภาพ PHR และ EMR

จากโครงสร้างที่ deploy แล้ว บทนี้เชื่อม journey ข้อมูลคลินิกระหว่างผู้ป่วยและแพทย์ อธิบายความต่าง PHR (ผู้ป่วยกรอก) กับ EMR (แพทย์ลงนาม)

วัตถุประสงค์ของบทนี้

ให้แพทย์และผู้ป่วยเข้าใจว่าใครเห็นข้อมูลอะไร และ API filter อะไรก่อนส่งให้ผู้ป่วย



ภาพที่ 9: 9. Health Records (PHR-EMR) (diagrams.drawio หน้า 9/12)

คำอธิบายและบริบท

PHR: ผู้ป่วยกรอก demographics, vital signs, allergies, medications ลง phr / vital_signs ใช้ JSONB ยึดหุ่นยนต์สำหรับฟิลด์เพิ่มเติม

แพทย์ดู PHR เมื่อมีสิทธิ (active appointment) ทุกครั้งบันทึก audit_logs

EMR: โครงสร้าง SOAP (Subjective, Objective, Assessment, Plan) AI สร้าง draft จาก meeting แพทย์แก้ไขใน EmrEditorChrome แล้ว sign

E-prescribing: medications JSONB + cds_warnings จาก Gemini/rules แพทย์ต้อง acknowledge warning ก่อนส่ง

Lab orders: สร้างจาก EMR ติดตามสถานะ ผล lab ใน results JSONB

Patient EMR view: API filter — ไม่ส่ง raw notes หรือ differential ให้ผู้ป่วย Timeline

รวมเหตุการณ์นัด PHR EMR instruction sheet ภาษาไทยง่ายหลัง validate

ประเด็นสำคัญ

- PHR: ผู้ป่วยเป็นเจ้าของข้อมูล self-entry
- EMR: แพทย์ลงนาม — MITL
- API: GET /api/phr, GET /api/emr/my (filter)
- CDS: คำเตือนยา — แพทย์ acknowledge
- Audit: ทุกการอ่าน PHR โดยแพทย์
- Timeline: รวมเหตุการณ์สุขภาพ

การอ่านแผนภาพ

แยก swimlane ผู้ป่วย vs แพทย์ และจุด INSERT/UPDATE ลงตาราง phr, emr, prescriptions

เชื่อมต่อบทถัดไป

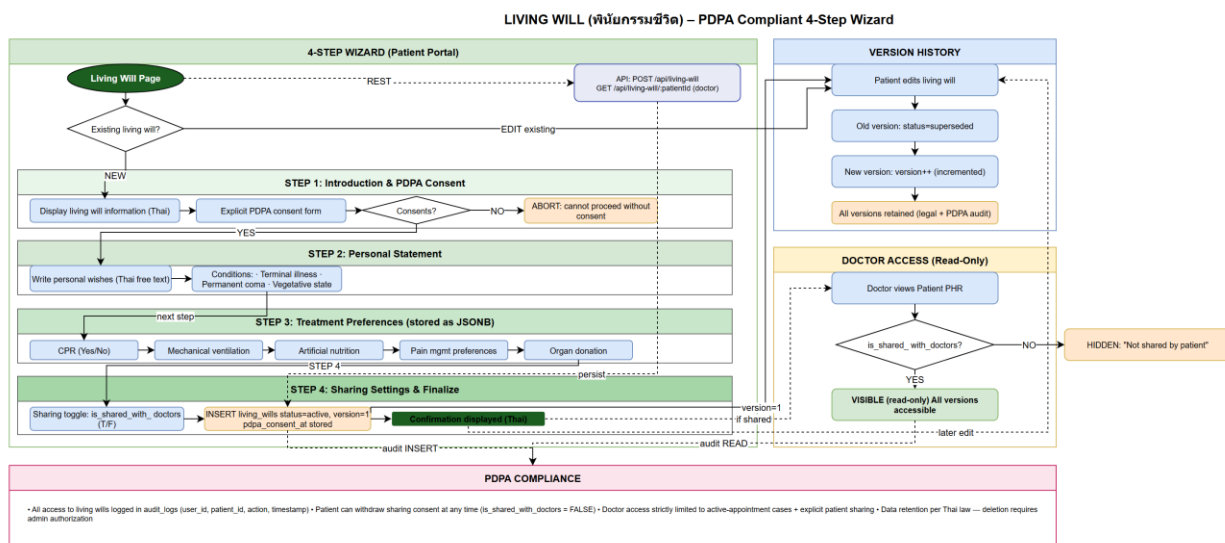
ภายใต้ PDPA หนังสือแสดงเจตนาชีวิตเป็นส่วนหนึ่งของข้อมูลผู้ป่วย — บทถัดไป

บทที่ 10 — พินัยกรรมชีวิต (Living Will)

ต่อจาก PHR/EMR บทนี้อธิบาย wizard PDPA และการแชร์ให้แพทย์อ่านแบบ read-only ข้อมูลอ่อนไหวต้องมี consent versioning และ audit ครบ

วัตถุประสงค์ของบทนี้

ให้ผู้ตรวจสอบ PDPA และแพทย์เข้าใจเงื่อนไขการเก็บ แก้ไข และเปิดเผย Living Will



ภาพที่ 10: 10. Living Will (diagrams.drawio หน้า 10/12)

คำอธิบายและบริบท

Wizard 4 ขั้น: (1) ความยินยอม PDPA (2) คำแถลงส่วนตัวภาษาไทย (3) ค่าการรักษา CPR/ventilator ฯลฯ ใน JSONB (4) ตั้งค่า is_shared_with_doctors

บันทึก living_wills status=active version เริ่มที่ 1 แก้ไขครั้งถัดไป supersede เวอร์ชันเก่า version++ ไม่ลบทางกายภาพ

แพทย์เห็นได้เมื่อ is_shared_with_doctors=true และมีสิทธิ์ดู PHR
มิฉะนั้นแสดงข้อความว่าไม่ได้รับการแชร์

ทุก read/write บันทึก audit_logs รองรับการตรวจสอบตามกฎหมาย

API: POST /api/living-will, GET สำหรับแพทย์ตาม patientId หน้า LivingWillPage ฝั่ง patient —
ไม่ยินยอม PDPA = หยุด wizard

ประเด็นสำคัญ

- 4 ขั้น wizard + PDPA consent
- Versioning — ไม่ hard delete
- แชร์แพทย์: is_shared_with_doctors
- Audit log ทุกการเข้าถึง
- pdpa_consent_at timestamp
- อ้างอิงกฎหมาย PDPA ไทย

การอ่านแผนภาพ

ตามลำดับขั้น wizard และเส้นแชร์ไปแพทย์ / version history

เชื่อมต่อบทถัดไป

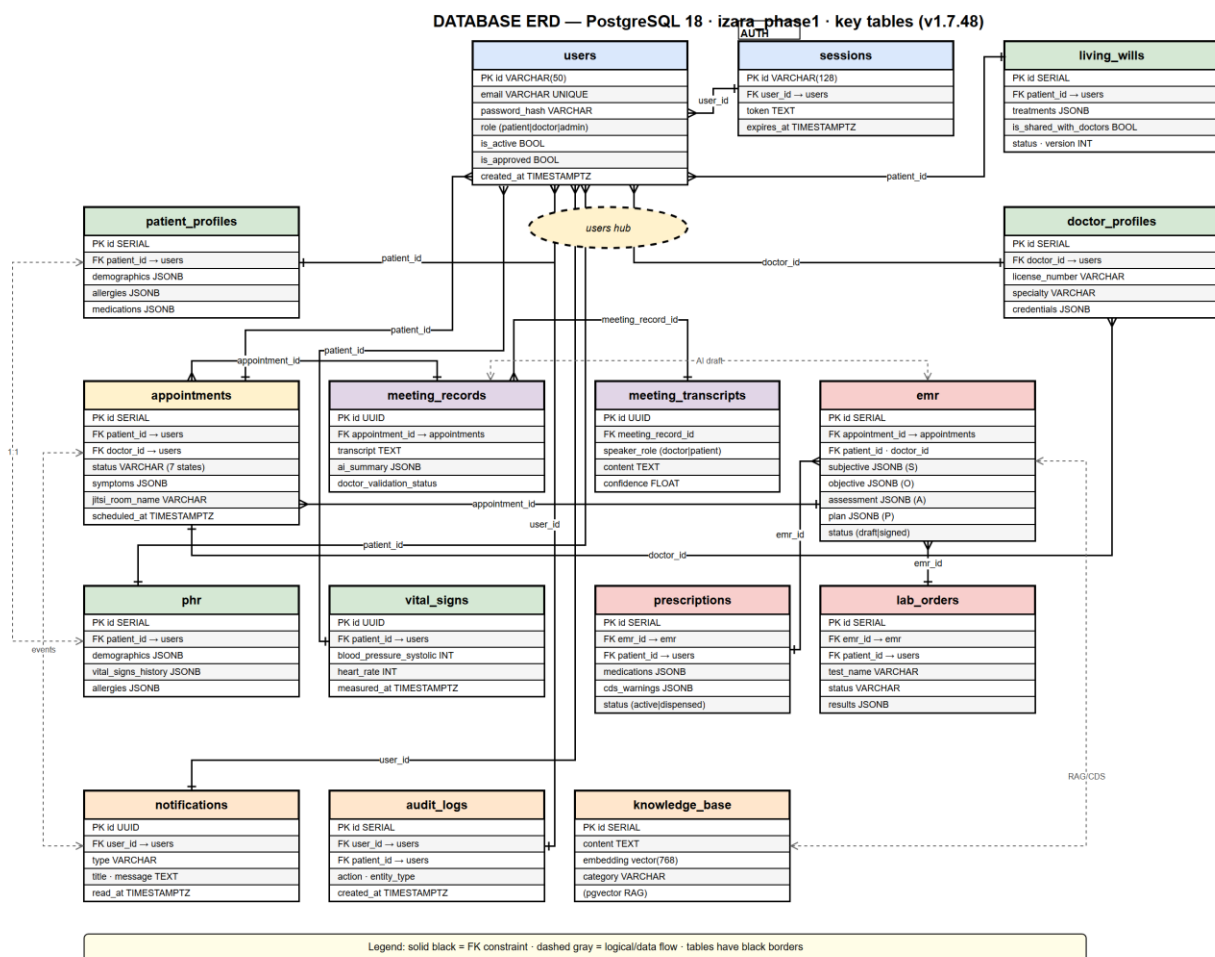
ข้อมูลทั้งหมดอยู่บน schema ที่บทถัดไปสรุปเป็น ERD

บทที่ 11 — ERD ฐานข้อมูล

สรุปความสัมพันธ์ตารางหลังจากอ่าน workflow ทั้งหมด — users เป็นศูนย์กลาง
แผนภาพใช้เส้นขอบสีดำและ layout ที่อ่าน FK ได้ชัด

วัตถุประสงค์ของบทนี้

อ้างอิงสำหรับนักพัฒนา DBA และผู้ตรวจสอบเมื่อเพิ่มฟีเจอร์หรือ migration



ภาพที่ 11: 11. Database ERD (diagrams.drawio หน้า 11/12)

คำอธิบายและบริบท

users เชื่อม patient_profiles, doctor_profiles, sessions, appointments, phr, notifications, audit_logs

appointments → meeting_records → meeting_transcripts appointments → emr → prescriptions, lab_orders

notifications.read_at ควบคุม isRead — ไม่ใช่คอล์มน์แยกโดยไม่ sync

knowledge_base.embedding vector(768) สำหรับ RAG ใน CDS และ AI Studio

living_wills แยก version ชัดเจน — ไม่ลบทางกายภาพเพื่อ audit

Schema ฉบับเต็ม: scripts/database/izara-database.sql — แผนภาพแสดงตารางหลัก Phase 1

GCE PostgreSQL 18 + pgvector extension — PK ส่วนใหญ่ SERIAL หรือ UUID

ประเด็นสำคัญ

- ศูนย์กลาง: users
- Clinical chain: appointments → emr → rx/lab
- Meeting chain: meeting_records → transcripts
- FK เส้นทึบ — เส้นประ = logical flow
- pgvector 768 มิติ
- Migration: scripts/database/migrations/

การอ่านแผนภาพ

มอง users กลางแผนภาพ แล้วตามกลุ่ม appointments/clinical และ notifications/audit

เชื่อมต่อบทถัดไป

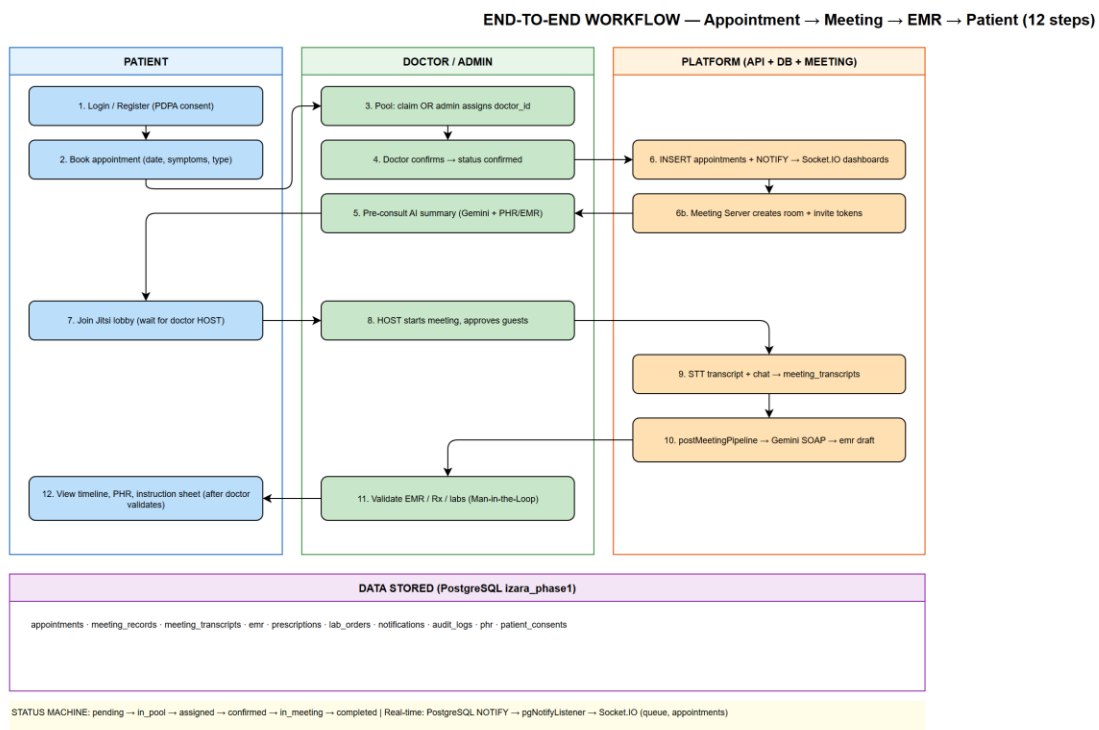
เมื่อ schema ชัดแล้ว บทสุดท้ายของ 12 แผนภาพคือ timeline E2E ที่เชื่อมทุกบทเข้าด้วยกัน

บทที่ 12 — ขั้นตอน E2E ทั้งเส้นทาง

บทปดของชุด 12 แผนภาพ — รวมทุกอย่างที่อธิบายมาเป็นลำดับเวลาเดียว สำหรับผู้ป่วย แพทย์ และระบบ ใช้เป็น checklist UAT และอบรม

วัตถุประสงค์ของบทนี้

ให้ผู้ทดสอบและผู้ใช้ใหม่เห็นภาพตั้งแต่ล็อกอินจนถึงดูผลรักษา สอดคล้อง Playwright groups A-P



ภาพที่ 12: 12. E2E Workflow Steps (diagrams.drawio หน้า 12/12)

คำอธิบายและบริบท

ขั้น 1-2: ล็อกอิน (patient/doctor) + PDPA consent ตามบทบาท

ขั้น 3-4: จองนัด กรอกอาการ → เข้า Pool หรือเลือกแพทย์

ขั้น 5-6: มอบหมาย/ยืนยัน → แจ้งเตือน realtime

ขั้น 7: แพทย์เตรียมตัว อ่าน PHR + AI triage

ชั้น 8–9: Join meeting (lobby → admit) บันทึก/ถอดความ

ชั้น 10–11: SOAP draft → แพทย์ sign EMR → prescriptions/lab

ชั้น 12: ผู้ป่วยดู timeline, PHR อัปเดต, instruction sheet ภาษาไทย

แผนภาพสอดคล้อง Playwright groups A–P ใน docs/screenshots/ — ใช้เป็นดัชนีทดสอบ UI

อ้างอิง Processes/FULL_WORKFLOW_CONTRACT.md

แบ่งช่วง: ก่อนพบ (1–7) ระหว่างพบ (8–9) หลังพบ (10–12) — จุดตัดสำคัญ Pool assign, Jitsi, EMR sign

ประเด็นสำคัญ

- Patient lane / Doctor lane / System lane
- 12 ขั้นตอน — ไม่ข้าม MITL
- UAT: docs/screenshots/group-A ถึง group-P
- จุดตัด: Pool, Jitsi admit, EMR sign
- หลัง sign เท่านั้นผู้ป่วยเห็นผล
- FULL_WORKFLOW_CONTRACT.md

ช่วง	ชั้น	ผลลัพธ์
ก่อนพบ	1–7	นัด confirmed, พร้อมประชุม
ระหว่างพบ	8–9	transcript, recording
หลังพบ	10–12	EMR signed, ผู้ป่วยได้รับแจ้ง

การอ่านแผนภาพ

อ่านจากบนลงล่างหรือซ้ายไปขวาตาม swimlane — ตรวจสอบว่าทุกชั้นมีในบท 1–11 แล้ว

บทสรุป

บทสรุป: รายงานฉบับนี้เชื่อมโยง 12 แผนภาพแรกจาก diagrams.drawio เป็นเรื่องราวเดียวเป็นภาษาไทย ตั้งแต่ภาพรวมพอร์ทัล สถาปัตยกรรม การยืนยันตัวตน นัดหมาย ประชุมและ AI รีเลย์ไทม์ การ deploy ข้อมูลคลินิก PDPA โครงสร้างฐานข้อมูล และเส้นทาง E2E —
โดยไม่แก้ไขไฟล์แผนภาพและไม่รวมแท็บที่ 13 เป็นต้นไป

ทีมพัฒนาเมื่อปรับโค้ดที่ส่งผลต่อสถาปัตยกรรม ควรอัปเดตแผนภาพใน draw.io ด้วยตนเอง
จากนั้นรัน `npm run diagrams:report` เพื่อ export PNG 12 แท็บแรกและส regenerate Word + PowerPoint ฉบับนี้

เอกสารที่เกี่ยวข้อง: docs/markdown/operations/URLS_AND_DEFAULT_USERS.md,
Processes/VIDEO_MEETING_JITSI_GEMINI.md,
scripts/database/izara-database.sql, Presentations/TECHNICAL_DOCUMENTATION.md